which means that the first argument passed to the function has to be a string, and that the value returned by `trim()` will also be a string. Here's what happens if we call the `trim()` function with a numerical argument:

```
$num = 123;
$res = trim($num);

var_dump($num);
var_dump($res);

Output:
int(123)
string(3) "123"
```

A stricter language would not allow numerical types to be used like this, but PHP simply converts `$num` to a string and `trim()` gets on with the job. Things aren't so magical for compound types, however. Let's try to call `trim()` with an array.

```
$myArr = ["string 1", "string 2"];
$res = trim($myArr);

Output:
Fatal error: Uncaught TypeError: trim(): Argument #1 ($string) must be of
type string, array given
```

As we can see, even if we give `trim()` an array of strings, PHP still gives us a fatal error. As for the other compound types, it is possible to build objects that can become strings when used as such; these objects can also be `callable` or `iterable`, but you'll need to write code to make them behave like that. Compound types (arrays and objects) cannot be coerced or cast without some intervention first. The same is true of `resource`. We'll cover that later in the book when we look at objects in more detail.

And what about `null`? Well, `null` is simply a single byte with a value of `0x00`. That's a valid ascii character (the "null byte") and therefore poses no problems to any code written to deal with string data. Passing `null` to the various string-handling functions in PHP will not cause any errors.